



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 内核线程

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 D501

实验时间: 2025 年 5 月 4 日

Section 1 实验概述

在本次实验中，我们将会学习到 C 语言的可变参数机制的实现方法。在此基础上，我们会揭开可变参数背后的原理，进而实现可变参数机制。实现了可变参数机制后，我们将实现一个较为简单的 printf 函数。此后，我们可以同时使用 printf 和 gdb 来帮助我们 debug。

本次实验另外一个重点是内核线程的实现，我们首先会定义线程控制块的数据结构——PCB。然后，我们会创建 PCB，在 PCB 中放入线程执行所需的参数。最后，我们会实现基于时钟中断的时间片轮转 (RR) 调度算法。在这一部分中，我们需要重点理解 asm_switch_thread 是如何实现线程切换的，体会操作系统实现并发执行的原理。

Section 2 预备知识与实验环境

该节总结实验需要用到的基本知识，以及主机型号、代码编译工具、重要三方库的版本号信息等。

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
 - 实验任务 1：完成混合编程的基本思路
 - 实验任务 2：完成使用 C/C++ 编写内核
 - 实验任务 3：完成中断的处理
 - 实验任务 4：完成时钟中断的处理

Section 3 实验任务

- 实验任务 1：printf 的实现
- 实验任务 2：线程的实现
- 实验任务 3：时钟中断的实现
- 实验任务 4：调度算法的实现

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：

学习可变参数机制，然后实现 printf 函数，你可以在材料中（src/3）的 printf 上进行改进，或者从头开始实现自己的 printf 函数。结果截图保存并说说你是怎么做的。

- 思路分析：

1. 理解并掌握 C 语言的可变参数机制

C 语言使用 `<stdarg.h>` 提供对可变参数的支持，关键宏包括：

`va_list`：定义一个指针类型的参数列表变量；

`va_start(ap, last_fixed_arg)`：初始化 `ap`，指向可变参数列表；

`va_arg(ap, type)`：依次读取每一个参数；

`va_end(ap)`：清理工作。

本实验中手动模拟了这套机制，通过 `typedef char* va_list` 和内存偏移实现，这是理解编译原理和栈帧布局的关键技术点。

2. 逐字符解析格式字符串 `fmt`

核心处理逻辑类似于状态机：

遇到普通字符：直接添加到输出缓冲区；

遇到 `%`：进入格式控制状态，读取下一个字符判断类型（如 `%d`，`%x`，`%s` 等）；

解析格式化数据时，从 `va_list` 中取出参数并转成字符串加入缓冲区。

3. 输出缓冲机制（提高效率，减少 I/O 次数）

设置一个固定大小的输出缓冲区（如 32 字节），每写入一个字符都判断是否溢出：

若未满：继续追加； 若满：输出整个缓冲区并清空。

这样做的目的是：提高输出效率，减少打印开销，模拟真实系统的行为。

4. 数字转换处理函数 `itos`

实现 `%d`，`%x`，`%o` 的关键是将数字转换成对应进制的字符串。你通过 `itos(char *numStr, uint32 num, uint32 mod)` 实现该功能，并用通用的倒序存储法转为字符串，这是一个模块化设计的好示例。

5. 错误处理和边界问题要注意：

传入负数时的处理（特别是 `%d` 格式）：需要手动加上负号；

`va_arg` 使用时要保证类型匹配，否则可能造成栈破坏；

格式字符串中 `%` 后没有合法控制符时的容错机制；

避免缓冲区溢出，始终确保末尾加上 `\0` 结束符。

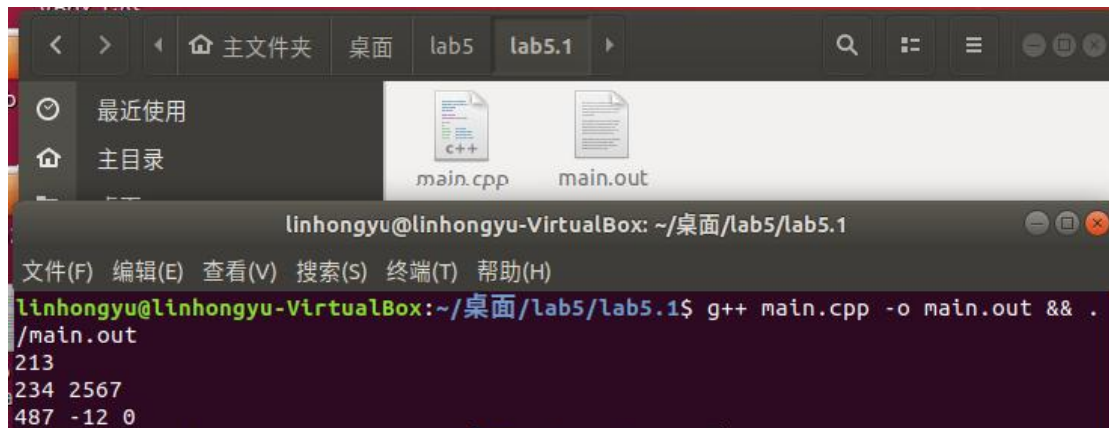
- 实验步骤:

1. 可变参数例子

- 1.1 例子

```
print_any_number_of_integers(1, 213);  
print_any_number_of_integers(2, 234, 2567);  
print_any_number_of_integers(3, 487, -12, 0);
```

- 1.1 结果展示



2. 宏定义

- 2.1 代码

```
typedef char *va_list;  
#define _INTSIZEOF(n) ((sizeof(n) + sizeof(int) - 1) & ~(sizeof(int) - 1))  
#define va_start(ap, v) (ap = (va_list)&v + _INTSIZEOF(v))  
#define va_arg(ap, type) (*(type *)((ap += _INTSIZEOF(type)) - _INTSIZEOF(type)))  
#define va_end(ap) (ap = (va_list)0)
```

- 2.2 发生错误

安装 gcc-multilib 和 g++-multilib。例如，在 ubuntu 下，使用如下命令安装

```
sudo apt install gcc-multilib g++-multilib
```

3. 实现 printf

- 3.1 printf 实现换行 ' \n ' 的输出字符串函数

实现 print 一个字符串的函数，对字符串每个字符进行遍历，当遇到 ' \n ' 时换行，其他情况直接输出该字符

```
int STDIO::print(const char *const str)  
{  
    int i = 0;  
  
    for (i = 0; str[i]; ++i)  
    {  
        switch (str[i])
```

```

{
    case '\n':
        uint row;

        row = getCursor() / 80;

        if (row == 24)
        {
            rollUp();
        }

        else
        {
            ++row;
        }

        moveCursor(row * 80);

        break;

    default:
        print(str[i]);

        break;

}

}

return i;
}
}

```

3.2 建立缓冲区

一次缓冲 32 个字节，缓冲区第 33 个字节空间存放字符串终止标识符 '\0'

```

int printf_add_to_buffer(char *buffer, char c, int &idx, const int BUF_LEN)
{
    int counter = 0;

    buffer[idx] = c;
    ++idx;

    if (idx == BUF_LEN)
    {
        buffer[idx] = '\0';
        counter = stdio.print(buffer);
        idx = 0;
    }

    return counter;
}

```

3.3 解析格式化输出参数

%d	按十进制整数输出
%c	输出一个字符

%s	输出一个字符串
%x	按 16 进制输出

3.4 printf 实现函数

原理：对 fmt 进行逐字符解析，对于每一个字符 fmt[i]，如果 fmt[i]不是%，则说明是普通字符，直接放到缓冲区即可。注意，将 fmt[i]放到缓冲区后可能会使缓冲区变满，此时如果缓冲区满，则将缓冲区输出并清空

完善：

3.4.1 添加八进制输出

```
case 'o':
{
    int temp = va_arg(ap, int);
    itos(number, temp, 8);
    for(int j=0; number[j]; ++j)
    {
        counter += printf_add_to_buffer(buffer, number[j], idx, BUF_LEN);
    }
    break;
}
```

3.4.2 添加代码块 {} 限制局部变量的作用域防止报错

```
case 'x':
{
    int temp = va_arg(ap, int);

    if (temp < 0 && fmt[i] == 'd')
    {
        counter += printf_add_to_buffer(buffer, '-', idx, BUF_LEN);
        temp = -temp;
    }
    itos(number, temp, (fmt[i] == 'd' ? 10 : 16));
    for (int j = 0; number[j]; ++j)
    {
        counter += printf_add_to_buffer(buffer, number[j], idx, BUF_LEN);
    }
    break;
}
```

3.4.3 添加输出代码

```
printf("LinHongYu_Pinrt: \"Ori:23320093 --> Oct:%o\\", 23320093);
```

3.4.4 完整代码见 code 文件

3.4.5 其中需要实现%d和%x的数字转换为对应的字符串函数

```

void itos(char *numStr, uint32 num, uint32 mod) {
    // 只能转换 2~26 进制的整数
    if (mod < 2 || mod > 26 || num < 0) {
        return;
    }

    uint32 length, temp;
    // 进制转换
    length = 0;
    while(num) {
        temp = num % mod;
        num /= mod;
        numStr[length] = temp > 9 ? temp - 10 + 'A' : temp + '0';
        ++length;
    }
    // 特别处理 num=0 的情况
    if(!length) {
        numStr[0] = '0';
        ++length;
    }
    // 将字符串倒转, 使得 numStr[0]保存的是 num 的高位数字
    for(int i = 0, j = length - 1; i < j; ++i, --j) {
        swap(numStr[i], numStr[j]);
    }

    numStr[length] = '\0';
}

```

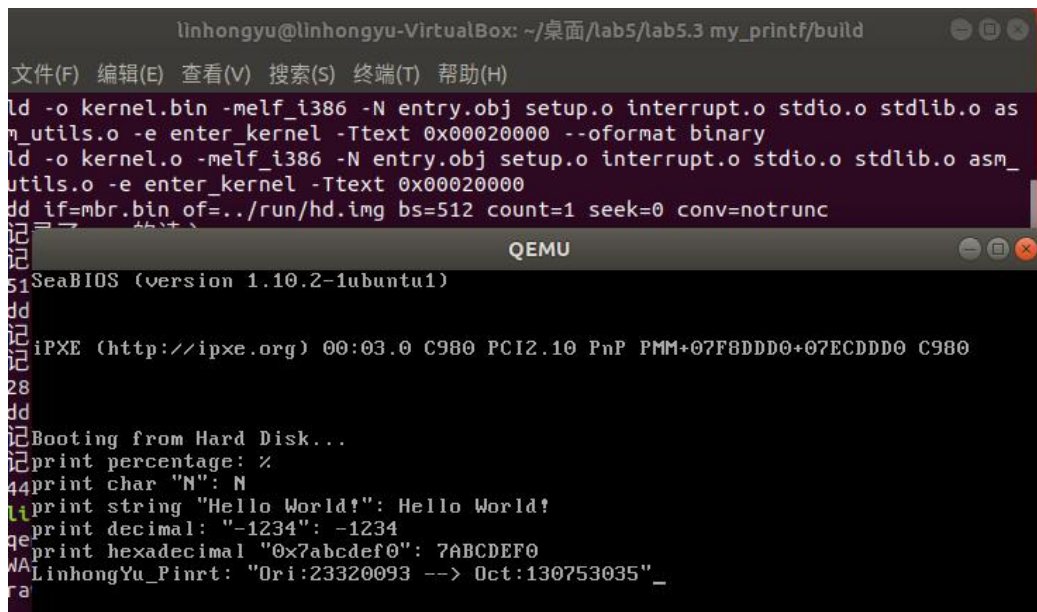
3.4.5 运行自己的 printf

```

Make
make run

```

- 实验结果展示:



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab5/lab5.3 my_printf/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
ld -o kernel.bin -melf_i386 -N entry.obj setup.o interrupt.o stdio.o stdlib.o as
m_utils.o -e enter_kernel -Ttext 0x00020000 --oformat binary
ld -o kernel.o -melf_i386 -N entry.obj setup.o interrupt.o stdio.o stdlib.o asm_
utils.o -e enter_kernel -Ttext 0x00020000
dd if=mbr.bin of=../run/hd.img bs=512 count=1 seek=0 conv=notrunc
=====
QEMU
SeaBIOS (version 1.10.2-1ubuntu1)
iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DD0+07ECDD0 C980
Booting from Hard Disk...
print percentage: %
print char "N": N
print string "Hello World!": Hello World!
print decimal: "-1234": -1234
print hexadecimal "0x7abcdef0": 7ABCDEF0
LinhongYu_Pintrt: "Ori:23320093 --> Oct:130753035" _
```

----- 实验任务 2 -----

- 任务要求：线程的实现

自行设计 PCB，可以添加更多的属性，如优先级等，然后根据你的 PCB 来实现线程，演示执行结果。

- 思路分析：

1. 本实验的核心是通过设计 PCB（进程控制块）实现线程管理。为支持线程调度，PCB 中增加了优先级、时间片等字段。

2. 通过 `executeThread` 函数创建线程，初始化 PCB 后加入就绪队列。

3. 每个线程函数输出自身 PID、优先级和名称，用于验证线程是否正确创建和调度。

4. 虽然未实现复杂调度策略，但通过优先级字段为后续扩展提供了基础。整体流程体现了线程的创建、上下文切换与基本调度机制。

- 实验步骤:

1. 使用优先级 priority

声明 PCB 结构体, 成员有栈指针, 线程名, 线程状态, 优先级, pid, 已执行时间等

```
struct PCB
{
    int *stack;
    char name[MAX_PROGRAM_NAME + 1];
    enum ProgramStatus status;
    int priority;           // 线程优先级
    int pid;               // 线程 pid
    int ticks;
    int ticksPassedBy;
    ListItem tagInGenerallist;
    ListItem tagInAlllist;
};
```

2. 调用代码

调用三个线程, 注意其对应的优先级 priority

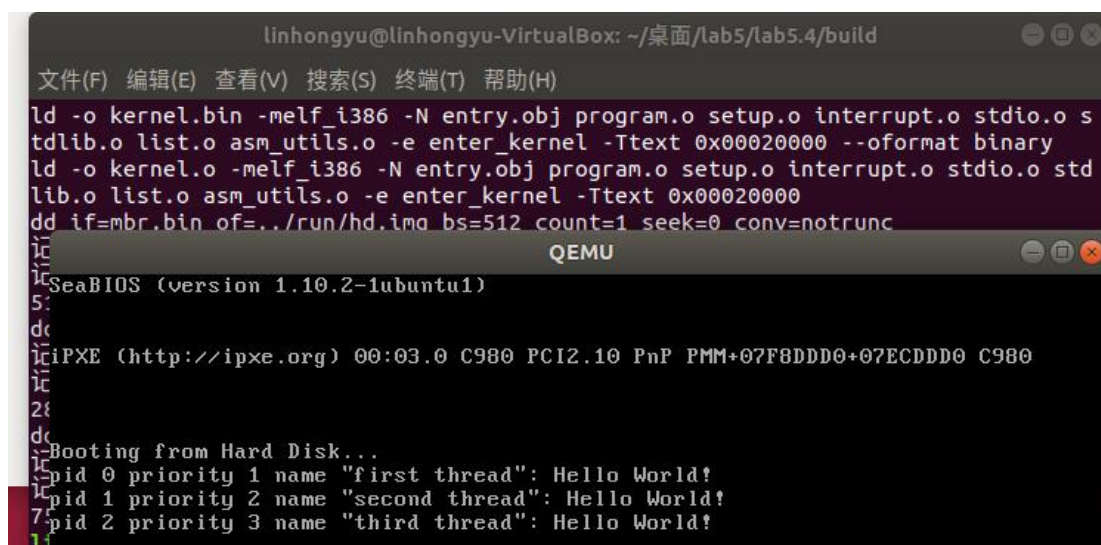
```
void third_thread(void *arg) {
    printf("pid %d priority %d name \"%s\": Hello World!\n",
programManager.running->pid, programManager.running->priority, programManager.running->name);
    while(1) {

    }
}

void second_thread(void *arg) {
    printf("pid %d priority %d name \"%s\": Hello World!\n",
programManager.running->pid, programManager.running->priority, programManager.running->name);
}

void first_thread(void *arg)
{
    // 第 1 个线程不可以返回
    printf("pid %d priority %d name \"%s\": Hello World!\n",
programManager.running->pid, programManager.running->priority, programManager.running->name);
    if (!programManager.running->pid)
    {
        programManager.executeThread(second_thread, nullptr, "second thread", 2);
        programManager.executeThread(third_thread, nullptr, "third thread", 3);
    }
    asm_halt();
}
```

- 实验结果展示：通过执行前述代码，可得下图结果。



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab5/lab5.4/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
ld -o kernel.bin -melf_i386 -N entry.obj program.o setup.o interrupt.o stdio.o s
tdlib.o list.o asm_utils.o -e enter_kernel -Ttext 0x00020000 --oformat binary
ld -o kernel.o -melf_i386 -N entry.obj program.o setup.o interrupt.o stdio.o std
lib.o list.o asm_utils.o -e enter_kernel -Ttext 0x00020000
dd if=mbr.bin of=../run/hd.img bs=512 count=1 seek=0 conv=notrunc
QEMU
SeaBIOS (version 1.10.2-1ubuntu1)
5:
dc
icIPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD0+07ECDDDD0 C980
ic
2ic
dc
icBooting from Hard Disk...
icpid 0 priority 1 name "first thread": Hello World!
icpid 1 priority 2 name "second thread": Hello World!
7cpid 2 priority 3 name "third thread": Hello World!
ic
```

----- 实验任务 3 -----

- 任务要求：时钟中断的实现

Assignment 3 时钟中断的实现

操作系统的线程能够并发执行的秘密在于我们需要中断线程的执行，保存当前线程的状态，然后调度下一个线程上处理机，最后使被调度上处理机的线程从之前被中断点处恢复执行。现在，同学们可以亲手揭开这个秘密。

编写若干个线程函数，使用gdb跟踪c_time_interrupt_handler、asm_switch_thread (eg: b c_time_interrupt_handler) 等函数，观察线程切换前后栈、寄存器、PC等变化，结合gdb、材料中“线程的调度”的内容来跟踪并说明下面两个过程。

1. 一个新创建的线程是如何被调度然后开始执行的。
2. 一个正在执行的线程是如何被中断然后被换下处理器的，以及换上处理器后又是如何从被中断点开始执行的。

通过上面这个练习，同学们应该能够进一步理解操作系统是如何实现线程的并发执行的。

- 思路分析：

1. 本实验通过设置时钟中断驱动线程调度。线程创建后由定时器周期性触发中断，在中断处理函数中调用调度器 schedule() 实现线程切换。
2. 调度器根据当前线程状态和时间片 ticks 判断是否切换线程。每个线程按优先级初始化时间片，ticks 每次中断减少，直至为 0 触发切换。
3. 通过 gdb 查看断点与寄存器状态，可验证调度逻辑与上下文切换的正确性。该机制是操作系统实现多线程并发的基础。

- 实验步骤：

1. 线程

first_thread 不可释放 asm_halt()

Third_thread 死循环 while(1)

```
void third_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
    while(1) {

    }
}

void second_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
}

void first_thread(void *arg)
{
    // 第1个线程不可以返回
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
    if (!programManager.running->pid)
    {
        programManager.executeThread(second_thread, nullptr, "second thread", 1);
        programManager.executeThread(third_thread, nullptr, "third thread", 1);
    }
    asm_halt();
}
```

2. 辅助调试的代码

```
void ProgramManager::schedule()
{
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();

    if (readyPrograms.size() == 0)
    {
        interruptManager.setInterruptStatus(status);
        return;
    }
    if (running->status == ProgramStatus::RUNNING)
    {
        running->status = ProgramStatus::READY;
        running->ticks = running->priority * 10;
        readyPrograms.push_back(&(running->tagInGenerallist));
    }
    else if (running->status == ProgramStatus::DEAD)
    {
    }
```

```

        releasePCB(running);
    }

    ListItem *item = readyPrograms.front();

    PCB *next = ListItem2PCB(item, tagInGenerallist);

    PCB *cur = running;

    next->status = ProgramStatus::RUNNING;

    running = next;

    readyPrograms.pop_front();

    asm_switch_thread(cur, next);

    interruptManager.setInterruptStatus(status);

    printf("The pid %d is ending,next pid is %d \n", cur->pid, next->pid);
}

```

3. 命令

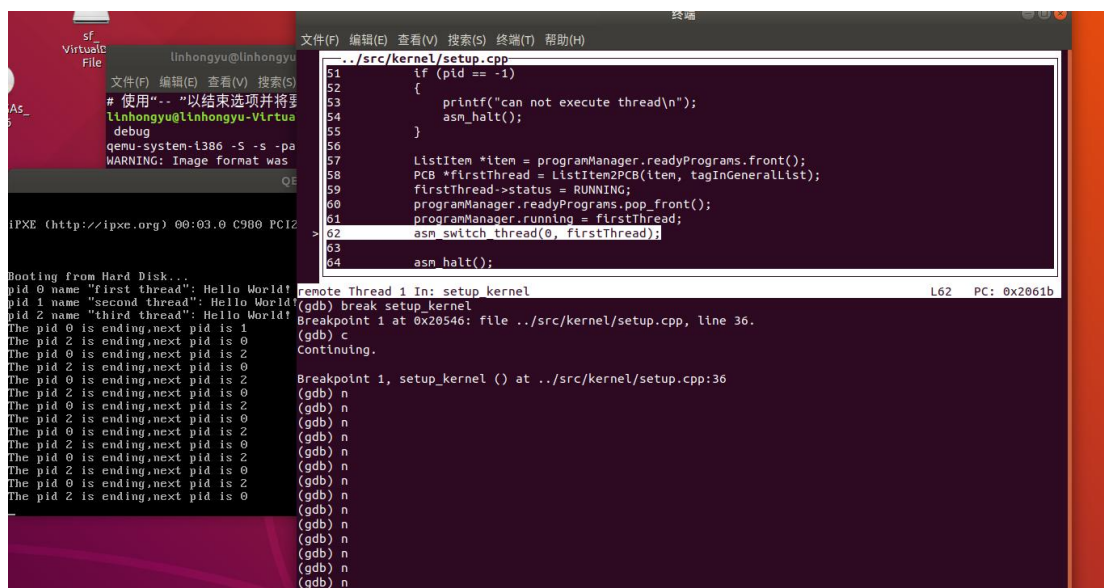
3.1 运行

```

make
Make run

```

3.2 观察结果



由图可知：一开始程序 1 结束了，最后只有程序 0 和程序 2 在运行

3.3 进入调试

```

Make debug

```

3.4 打断点

```

(gdb)break c_time_interrupt_handler
(gdb)break schedule

```

3.5 查看 ticks 信息

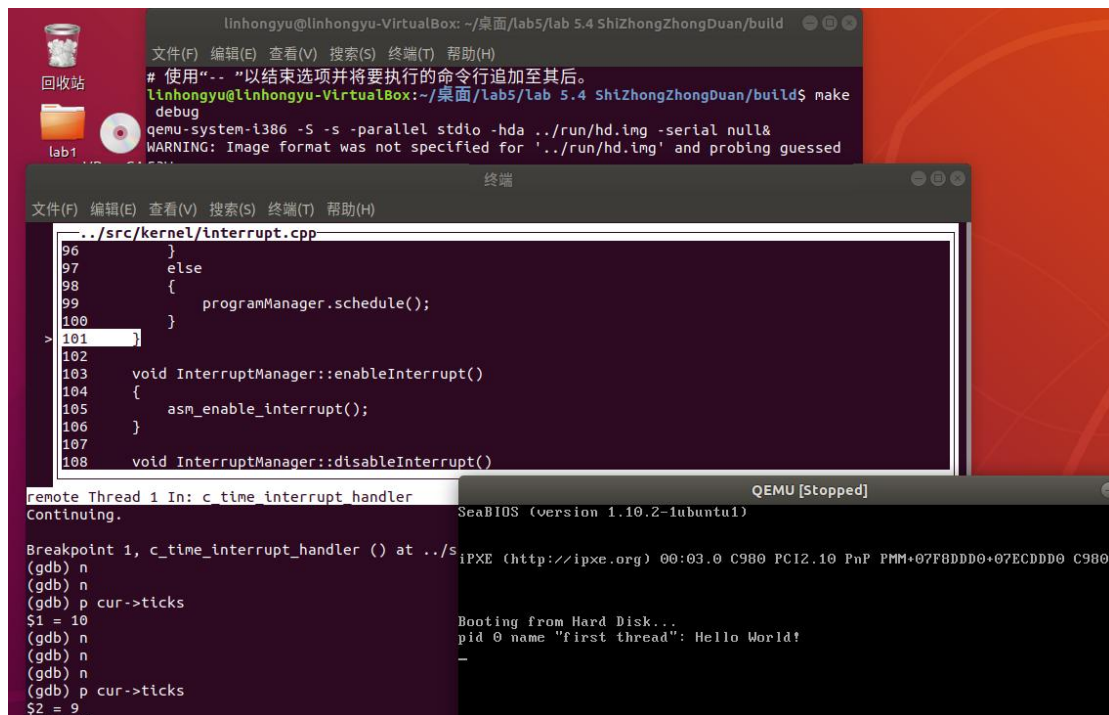
```

(gdb)p cur->ticks

```

3.6 观察结果

3.6.1 ticks 从 10 递减到 0



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab5/lab 5.4 ShiZhongZhongDuan/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
# 使用"--"以结束选项并将要执行的命令行追加至其后。
linhongyu@linhongyu-VirtualBox:~/桌面/lab5/lab 5.4 ShiZhongZhongDuan/build$ make
debug
qemu-system-i386 -S -s -parallel stdio -hda ../run/hd.img -serial null&
WARNING: Image format was not specified for '../run/hd.img' and probing guessed

终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
../src/kernel/interrupt.cpp
96     }
97     else
98     {
99         programManager.schedule();
100    }
> 101
102
103    void InterruptManager::enableInterrupt()
104    {
105        asm_enable_interrupt();
106    }
107
108    void InterruptManager::disableInterrupt()

remote Thread 1 In: c_time_interrupt_handler
Continuing.
Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) n
(gdb) n
(gdb) p cur->ticks
$1 = 10
(gdb) n
(gdb) n
(gdb) n
(gdb) p cur->ticks
$2 = 9

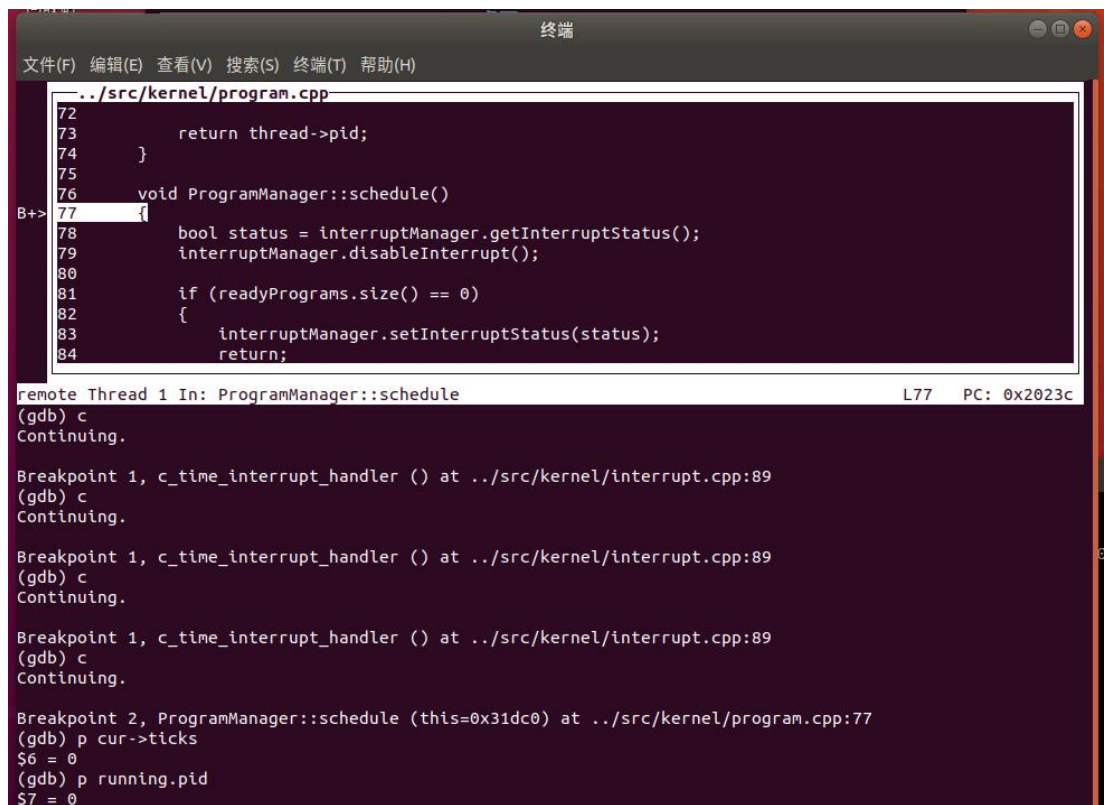
QEMU [Stopped]
SeaBIOS (version 1.10.2-lubuntu1)
iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DD0+07ECDD0 C980
Booting from Hard Disk...
pid 0 name "first thread": Hello World!
```

```
Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) p cur->ticks
$3 = 8
(gdb) c
Continuing.

Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) p cur->ticks
$4 = 7
(gdb) c
Continuing.

Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) p cur->ticks
$5 = 6
(gdb)
```

3.6.2 Ticks=0 时进入调试，此时 pid=0



```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/kernel/program.cpp
72
73     return thread->pid;
74 }
75
76 void ProgramManager::schedule()
B-> 77 {
78     bool status = interruptManager.getInterruptStatus();
79     interruptManager.disableInterrupt();
80
81     if (readyPrograms.size() == 0)
82     {
83         interruptManager.setInterruptStatus(status);
84         return;
85     }

remote Thread 1 In: ProgramManager::schedule L77 PC: 0x2023c
(gdb) c
Continuing.

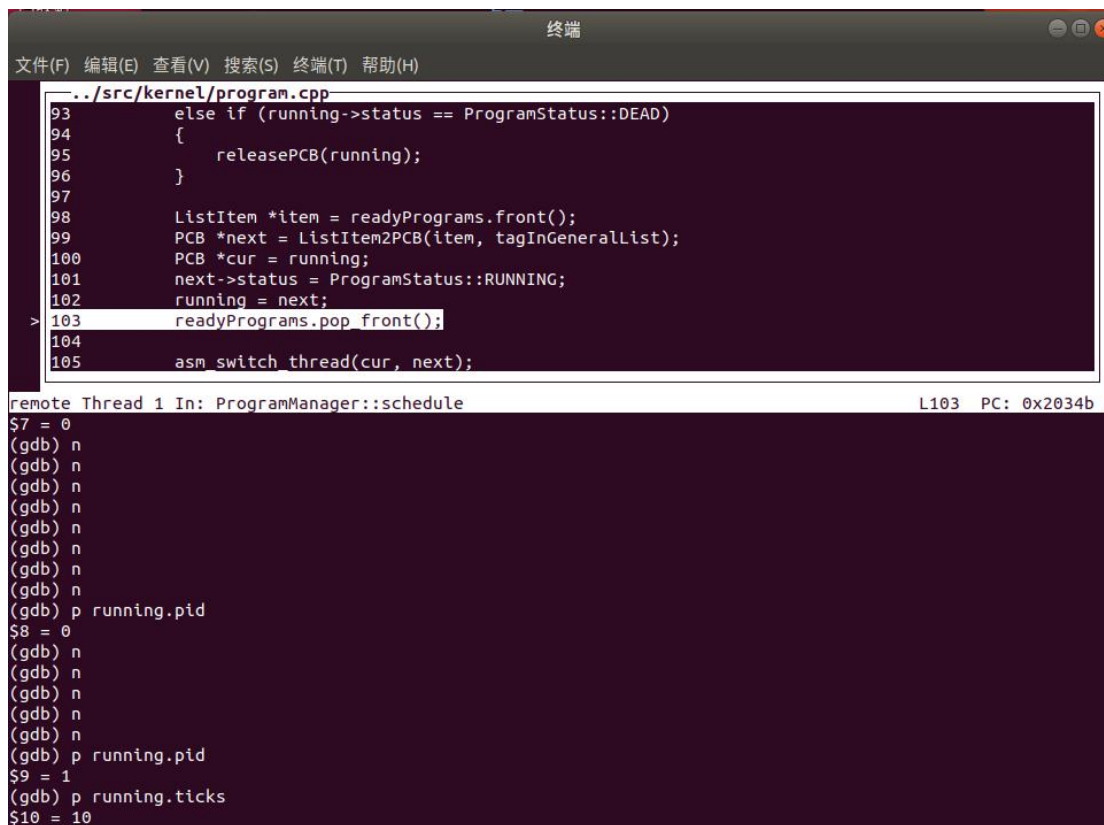
Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) c
Continuing.

Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) c
Continuing.

Breakpoint 1, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:89
(gdb) c
Continuing.

Breakpoint 2, ProgramManager::schedule (this=0x31dc0) at ../src/kernel/program.cpp:77
(gdb) p cur->ticks
$6 = 0
(gdb) p running.pid
$7 = 0
```

3.6.3 由图可知：切换 runing 进程，Pid=1，Ticks=10 完成调度



```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

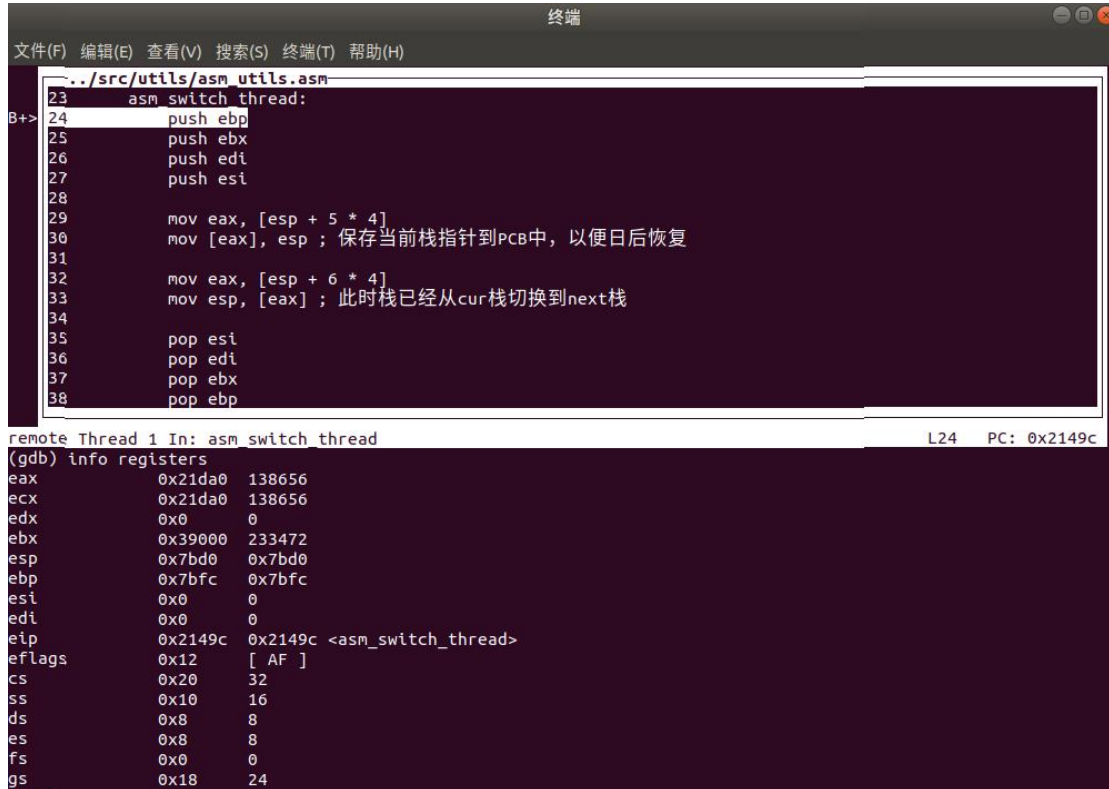
../src/kernel/program.cpp
93     else if (running->status == ProgramStatus::DEAD)
94     {
95         releasePCB(running);
96     }
97
98     ListItem *item = readyPrograms.front();
99     PCB *next = ListItem2PCB(item, tagInGeneralList);
100     PCB *cur = running;
101     next->status = ProgramStatus::RUNNING;
102     running = next;
> 103     readyPrograms.pop_front();
104
105     asm switch_thread(cur, next);

remote Thread 1 In: ProgramManager::schedule L103 PC: 0x2034b
$7 = 0
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) p running.pid
$8 = 0
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) p running.pid
$9 = 1
(gdb) p running.ticks
$10 = 10
```


3.6.4 查看寄存器信息

```
break asm_switch_thread  
info registers
```

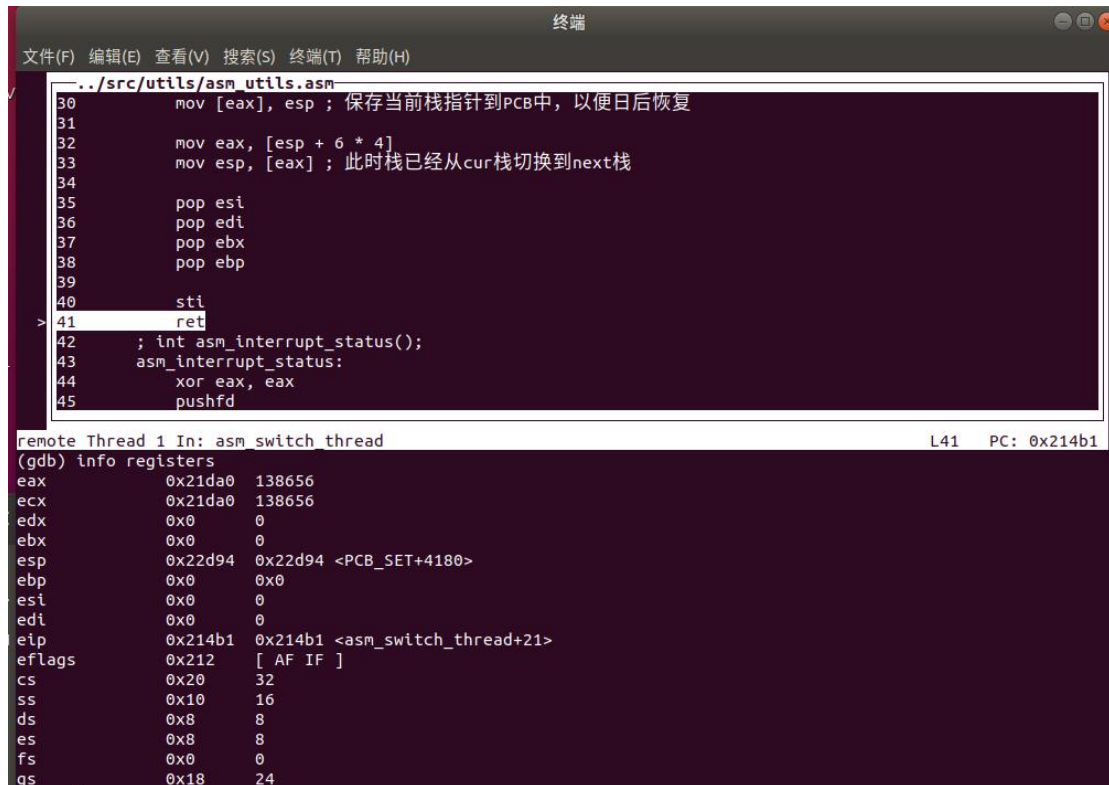
执行前，查看寄存器信息



The screenshot shows a debugger window with a menu bar (File, Edit, View, Search, Terminal, Help) and a toolbar. The main window displays assembly code from `../src/utils/asm_utils.asm`. The code includes comments in Chinese. The register window at the bottom shows the state of various registers.

```
remote Thread 1 In: asm_switch_thread  
(gdb) info registers  
eax      0x21da0 138656  
ecx      0x21da0 138656  
edx      0x0    0  
ebx      0x39000 233472  
esp      0x7bd0 0x7bd0  
ebp      0x7bfc 0x7bfc  
esi      0x0    0  
edi      0x0    0  
eip      0x2149c 0x2149c <asm_switch_thread>  
eflags   0x12   [ AF ]  
cs       0x20   32  
ss       0x10   16  
ds       0x8    8  
es       0x8    8  
fs       0x0    0  
gs       0x18   24
```

执行后，查看寄存器信息



The screenshot shows the same debugger window after execution. The assembly code is now at a later point, and the register values have changed. The instruction pointer (eip) is now at `0x214b1`.

```
remote Thread 1 In: asm_switch_thread  
(gdb) info registers  
eax      0x21da0 138656  
ecx      0x21da0 138656  
edx      0x0    0  
ebx      0x0    0  
esp      0x22d94 0x22d94 <PCB_SET+4180>  
ebp      0x0    0x0  
esi      0x0    0  
edi      0x0    0  
eip      0x214b1 0x214b1 <asm_switch_thread+21>  
eflags   0x212   [ AF IF ]  
cs       0x20   32  
ss       0x10   16  
ds       0x8    8  
es       0x8    8  
fs       0x0    0  
gs       0x18   24
```

- 实验结果展示：

运行结果已经在实验步骤中逐步展示

----- 实验任务 4 -----

- 任务要求：调度算法的实现

在材料中，我们已经学习了如何使用时间片轮转算法来实现线程调度。但线程调度算法不止一种，例如：

先来先服务

最短作业（进程）优先

响应比最高优先算法

优先级调度算法

多级反馈队列调度算法此外，我们的调度算法还可以是抢占式的。

现在，同学们需要将线程调度算法修改为上面提到的算法或者是同学们自己设计的算法。然后，同学们需要自行编写测试样例来呈现你的算法实现的正确性和基本逻辑。最后，将结果截图并说说你是怎么做的。

参考资料：<https://zhuanlan.zhihu.com/p/97071815>

Tips:

先来先服务最简单。

有些调度算法的实现可能需要用到中断。

- 思路分析：

1. 调度器需按顺序从就绪队列中取出线程，不考虑优先级或时间片；
2. 线程一旦开始运行直至退出，不被中断（非抢占式）；
3. 修改 `program_exit()`，当所有线程运行完毕后进入 `halt`，避免无限阻塞；
4. 避免使用 `asm_halt()` 阻塞主线程，确保其能自然退出，释放控制权给调度器。
5. 需注意线程不能手动阻塞或进入死循环，否则将影响调度流程的正确呈现。通过输出信息观察执行顺序即可验证算法正确性。

- 实验步骤:

1. 中断处理函数空实现

```
extern "C" void c_time_interrupt_handler()
{
}
}
```

2. 程序退出逻辑修改

判断条件 `programManager.readyPrograms.empty()`

当等待队列为非空时继续执行

```
void program_exit()
{
    PCB *thread = programManager.running;
    thread->status = ProgramStatus::DEAD;
    if (!programManager.readyPrograms.empty())
    {
        programManager.schedule();
    }
    else
    {
        interruptManager.disableInterrupt();
        printf("halt\n");
        asm_halt();
    }
}
```

3. 线程代码

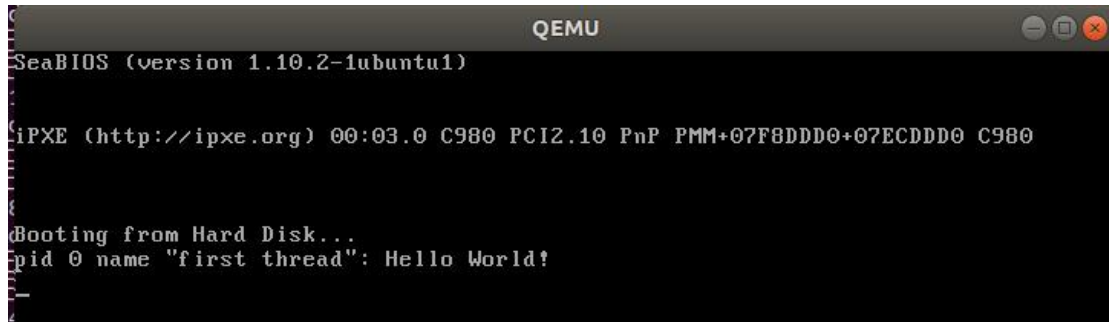
```
void third_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
}

void second_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
}

void first_thread(void *arg)
{ // 第1个线程不可以返回
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
    if (!programManager.running->pid)
    {
        programManager.executeThread(second_thread, nullptr, "second thread", 2);
        programManager.executeThread(third_thread, nullptr, "third thread", 3);
    }
    asm_halt();
}
```

4. 上诉程序运行结果

First Thread 处于 amsm_halt, 无法结束释放, 因此其他线程无法执行



```
SeaBIOS (version 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD0+07ECDDDD0 C980

Booting from Hard Disk...
pid 0 name "first thread": Hello World!
```

5. 线程代码



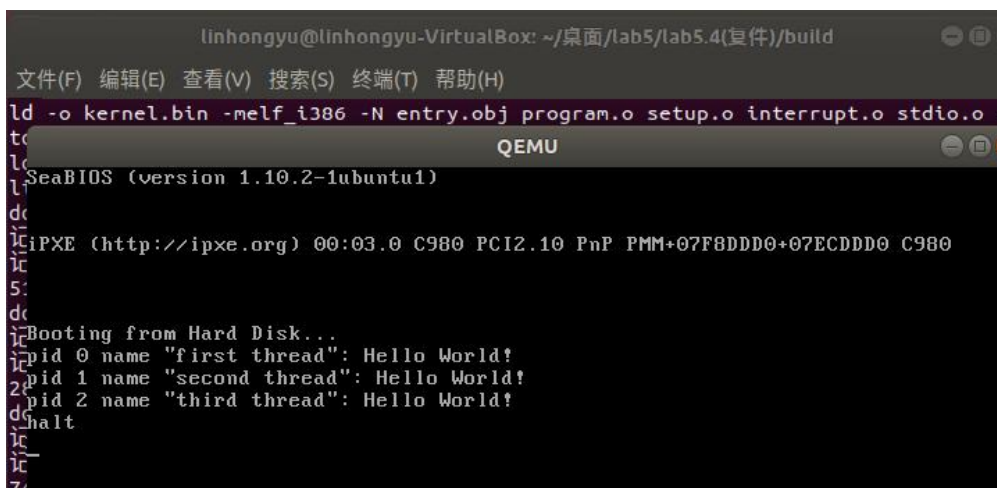
```
void third_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
}

void second_thread(void *arg) {
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
}

void first_thread(void *arg)
{
    // 第1个线程不可以返回
    printf("pid %d name \"%s\": Hello World!\n", programManager.running->pid, programManager.running->name);
    if (!programManager.running->pid)
    {
        programManager.executeThread(second_thread, nullptr, "second thread", 2);
        programManager.executeThread(third_thread, nullptr, "third thread", 3);
    }
}
```

6. 上诉程序运行结果

三个线程都能顺利执行, 并最后没有等待的线程, 输出” halt”



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab5/lab5.4(复件)/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
ld -o kernel.bin -melf_i386 -N entry.obj program.o setup.o interrupt.o stdio.o s
tc
QEMU
SeaBIOS (version 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD0+07ECDDDD0 C980

Booting from Hard Disk...
pid 0 name "first thread": Hello World!
pid 1 name "second thread": Hello World!
pid 2 name "third thread": Hello World!
halt
```

对比两次结果可知，先到先服务算法的作用

- 实验结果展示：

实验结果在上面的过程中逐步展示

Section 5 实验总结与心得体会

1. 实现 printf 函数：掌握可变参数机制与底层实现原理

深入研究了 C 语言的可变参数机制，理解了 `va_list`、`va_start`、`va_arg` 和 `va_end` 的原理，并通过模拟其底层实现方式，提升了对函数调用和栈帧布局的理解。随后我们完成了一个简化版的 `printf` 函数，实现了对 `%d`、`%s`、`%c`、`%x` 乃至 `%o` 等格式控制符的支持，并设计了缓冲区机制以提高输出效率。

2. 内核线程机制：PCB 设计与线程创建

设计并实现了线程控制块（PCB）结构，添加了如线程名、优先级、PID、状态等关键字段，从而支撑内核线程的完整生命周期管理。通过调用 `executeThread` 创建线程、初始化 PCB 并加入就绪队列，我们能够在屏幕上看到每个线程的执行信息，从而验证线程调度逻辑的正确性。

3. 调度与时钟中断：从时间片轮转到非抢占式调度策略

实验的后续部分着眼于线程调度与中断管理。我们首先实现了基于时钟中断驱动的时间片轮转（RR）调度算法，并通过 GDB 调试验证了 `ticks` 值的递减与线程切换行为。在此基础上，我们尝试修改调度策略为先来先服务（FCFS），将其改为非抢占式，并调整程序退出逻辑以避免阻塞主线程。通过这些实验，我们切实体会到调度算法在操作系统中的核心作用，也认识到不同策略的实现难度和适用场景差异。